

Objetivo del tema

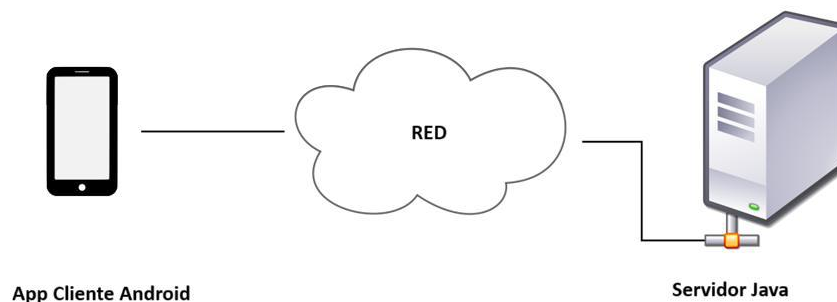
En este tema aprenderemos a comunicar una App con un Servidor mediante TCP utilizando Socket.io.

1.1 Web Services y Retrofit

Socket.IO es una biblioteca basada en eventos para aplicaciones web en tiempo real. Permite la comunicación bidireccional entre clientes y servidores web. Consta de dos componentes: un cliente y un servidor. Socket.IO también es un protocolo, donde diferentes implementaciones compatibles del protocolo pueden comunicarse entre sí. Por ejemplo, hay implementaciones para Node.js, Deno (JavaScript), C++, Java, Python, Swift y por supuesto, Kotlin.

Socket.IO utiliza principalmente el protocolo WebSocket, que a su vez es un protocolo de comunicaciones que proporciona un canal bidireccional y simultáneo a través de una única conexión de tipo **TCP**.

Vamos a programar un ejemplo simple que permita comunicación cliente-servidor mediante Socket.io. Aunque ambos utilizarán la misma biblioteca, verás que la implementación será diferente para ambos (aunque similar). Vamos a suponer que disponemos de una **App Cliente Android** que solicita al **Servidor Java** la información referente a unos alumnos. El esquema general de la aplicación será:



1.2 Diseñando las operaciones

Antes de ponerse a picar código, hay que pararse a diseñar las operaciones vamos a realizar contra el servidor. En nuestro ejemplo vamos a suponer que nuestra **App Cliente Android** necesita obtener del **Servidor Java** la información de un alumno (login); el listado de todos los alumnos (selectAll); y comunicar que se ha deslogueado (logout).

Partiendo de esta base, podemos empezar a diseñar la lógica detrás de estas operaciones.

Login	
Nombre del evento:	“onLogin”
Parámetros enviados	userName
Parámetros devueltos:	Alumno
Propósito:	El servidor devuelve el alumno de Base de Datos cuyo userName coincide con el solicitado.

GetAll	
Nombre del evento:	“onGetAll”
Parámetros enviados	-
Parámetros devueltos:	Listado de Alumnos
Propósito:	El servidor devuelve todos los alumnos de la Base de Datos.

Logout	
Nombre del evento:	“onLogout”
Parámetros enviados	userName
Parámetros devueltos:	-
Propósito:	El servidor toma nota de que el cliente se ha deslogueado.

El apartado **nombre del evento** es importante. Socket.io utiliza ese nombre para distinguir entre diferentes solicitudes al servidor. Por tanto, utilizar el nombre del evento incorrecto puede llevar a errores.

1.3 El Servidor

El Servidor ya lo tienes creado en la carpeta de ejemplos: **AlumnoServer-SocketIO**. Vamos a detallar aquí los pasos que se han seguido para crearlo desde cero.

Paso 1 – Proyecto Maven: En Eclipse, creamos un **Proyecto Maven**. Le indicamos que el nivel de compilación sea Java 20 y el de ejecución sea el mismo. Esto se debe a que estaremos utilizando operaciones lambda que requieres una jre 1.8 o superior.

Paso 2 – Dependencias: Le añadimos las siguientes dependencias al proyecto en el **pom.xml**.

```
<!-- Socket.io dependencies -->

<dependency>
  <groupId>com.corundumstudio.socketio</groupId>
  <artifactId>netty-socketio</artifactId>
  <version>2.0.3</version>
</dependency>

<!-- GSON dependencies -->

<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.1</version>
</dependency>
```

Netty es una implementación open-source de socket-io; y gson es una librería de Google para trabajar con JSON.

Nota: en el código de ejemplo verás que hay una serie de dependencias adicionales para intentar evitar que se genere un warning molesto al arrancar el servidor. No son necesarias.

Paso 3 – Estructura: Genera la estructura de clases y paquetes tal y como sigue:

```

  ▾ alumnoServer
    ▾ src/main/java
      ▾ com.alumno.server.alumnoServer
        > App.java
      ▾ com.alumno.server.alumnoServer.socketIO
        > SocketIOModule.java
      ▾ com.alumno.server.alumnoServer.socketIO.config
        > Events.java
      ▾ com.alumno.server.alumnoServer.socketIO.model
        > MessageInput.java
        > MessageOutput.java
```

A grandes rasgos:

- **App:** Contiene el main de la aplicación
- **SocketIOModule:** Contiene el código principal de comunicación
- **Events:** Listado de los eventos que escuchará y emitirá el servidor.
- **MessageInput / MessageOutput:** Lo que se envía y recibe el servidor.

Paso 4 – Mensajes: Las clases **MessageInput** y **MessageOutput** definen lo que enviamos y recibimos de la App en nuestro servidor. En nuestro caso es exactamente lo mismo: un texto plano (un JSON). Por tanto, ambas clases serán idénticas. Para simplificar las cosas, creamos una clase **AbstractMessage** con los campos necesarios de la que heredarán ambas.

Nosotros no nos tendremos que preocupar de convertir un JSON en un Objeto Java (o viceversa) porque esa conversión la hace automáticamente la librería **netty-socket.io**

```
public abstract class AbstractMessage {  
  
    private String message = null;  
  
    public AbstractMessage(String message) {  
        super();  
        this.message = message;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
  
    public void setMessage(String message) {  
        this.message = message;  
    }  
}  
  
public class MessageInput extends AbstractMessage {  
  
    public MessageInput(String message) {  
        super(message);  
    }  
  
    public class MessageOutput extends AbstractMessage {  
  
        public MessageOutput(String message) {  
            super(message);  
        }  
    }  
}
```

Paso 5 – Eventos: la clase **Events** es una enumeración en la que tendremos que indicar los eventos que genera o responde nuestra app. En nuestro caso son tres, indicados más arriba, y dos respuestas al cliente. Por tanto:

```
public enum Events {  
  
    ON_LOGIN ("onLogin"),  
    ON_GET_ALL ("onGetAll"),  
    ON_LOGOUT ("onLogout"),  
    ON_LOGIN_ANSWER ("onLoginAnswer"),  
    ON_GET_ALL_ANSWER ("onGetAllAnswer");  
  
    public final String value;  
  
    private Events(String value) {  
        this.value = value;  
    }  
}
```

Paso 6 – El Module: **SocketIOModule** es la clase que va a definir el comportamiento general de la comunicación entre cliente y servidor. Dado que la comunicación se base en eventos, en pocas palabras puede decirse que, cada vez que el cliente ‘hace algo’ el servidor ‘hace algo al respecto’. Si decide hacer A el server hace A. En nuestro caso, si el cliente decide hacer login el servidor ejecutará el método login ().

Podemos decir que hay dos tipos de eventos: los predefinidos y los que nosotros podemos definir. Los predefinidos son **onConnect** y **onDisconnect**. No hay que indicarlos en la clase Events porque siempre existen. Lo único que habrá que hacer es definir lo que queremos que haga el Server cuando los recibe. Los otros son, como habrás adivinado, los que hemos definido en la clase Events.

```

public class SocketIOModule {

    // The server
    private SocketIOServer server = null;

    public SocketIOModule(SocketIOServer server) {
        super();
        this.server = server;

        // Default events (for control the connection of clients)
        server.addConnectListener(onConnect());
        server.addDisconnectListener(onDisconnect());
    }

    private ConnectListener onConnect() {
        return (client -> {
            client.joinRoom("default-room");
            System.out.println("New connection, Client: " + client.getRemoteAddress());
        });
    }

    private DisconnectListener onDisconnect() {
        return (client -> {
            client.leaveRoom("default-room");
            System.out.println(client.getRemoteAddress() + " disconnected from server");
        });
    }
}

```

El siguiente código lo que hace es definir el comportamiento del Sever cuando un cliente se conecta y se desconecta. Si te fijas, lo único que hacemos es añadir o eliminar al cliente de una **Room** y escribir una traza del suceso. Las Room (salas) son agrupaciones de clientes. Esto permite enviar y recibir mensajes entre ellos, de forma similar a una sala de chat. No vamos a usar esta característica en este proyecto.

Ahora, hay que **añadir el comportamiento** para los eventos que nosotros hemos definido. La forma de hacerlo es similar: en el constructor definimos los eventos y los métodos que se lanzan, y después definimos dichos métodos. Obviamente, no incluimos las respuestas.

Añadimos al constructor:

```

// Custom events
server.addEventListener(Events.ON_LOGIN.value, MessageInput.class, this.login());
server.addEventListener(Events.ON_GET_ALL.value, MessageInput.class, this.getAll());
server.addEventListener(Events.ON_LOGOUT.value, MessageInput.class, this.logout());

```

Y añadimos los métodos vacíos con una traza para hacer el seguimiento:

```

private DataListener<MessageInput> login() {
    return ((client, data, ackSender) -> {
        System.out.println("Client from " + client.getRemoteAddress() + " wants to login");
    });
}

```

Ahora podemos empezar a añadir el código del comportamiento de cada evento.

Paso 7 – Lógica del Login: Vamos a implementar **el Login**. Se ha decidido que el usuario va a enviarnos el userName y tendremos que devolverle el Alumno correspondiente desde Base de Datos. Para nuestro ejemplo, que NO USA BBDD, vamos añadir en un paquete aparte la clase Alumno, que obviamente es una Entidad, y luego simularemos que la BBDD funciona correctamente.

```
public class Alumno {  
  
    private int id = 0;  
    private String name = null;  
    private String surname = null;  
    private String pass = null;  
    private int edad = 0;  
}
```

Vamos ahora al método login () y completamos el código. **Data** contiene la información que nos ha llegado del cliente, y es un **MessageInput**. De él podemos extraer el mensaje, que está en formato JSON. Para poder usarlo, hay que parsearlo mediante GSON. Obviamente, nos tocará averiguar exactamente qué nos está llegando desde el cliente para poder parsear correctamente. En nuestro caso asumimos que llega esto: {"userName": "patata"}.

Extraemos el userName y vamos a base de datos en busca de ese Alumno. Esto nos lo saltamos, y devolvemos cualquier cosa. Finalmente, convertimos Alumno en un JSON y se lo mandamos de vuelta al cliente dentro de un **MessageOutput**. Fíjate que el evento de vuelta es un **loginAnswer**. No lo implementamos de evento en el Server porque el Cliente NO NOS ENVÍA un 'answer' nunca.

Por cierto... para responder con errores simplemente se añade un if () y se envía un evento **errorAnswer** con otro JSON diferente. Por si te lo preguntabas.

Nos queda algo así:

```
private DataListener<MessageInput> login() {  
    return ((client, data, ackSender) -> {  
        System.out.println("Client from " + client.getRemoteAddress() + " wants to login");  
  
        // The JSON message from MessageInput  
        String message = data.getMessage();  
  
        // We parse the JSON into an JsonObject  
        // The JSON should be something like this: {"userName": "patata"}  
        Gson gson = new Gson();  
        JsonObject jsonObject = gson.fromJson(message, JsonObject.class);  
        String userName = jsonObject.get("userName").getAsString();  
  
        // We access to database and...  
        // Let's say it answers with this...  
        Alumno alumno = new Alumno(0, userName, "potato", "pass", 20);  
  
        // We parse the answer into JSON  
        String answerMessage = gson.toJson(alumno);  
  
        // ... and we send it back to the client inside a MessageOutput  
        MessageOutput messageOutput = new MessageOutput(answerMessage);  
        client.sendEvent(Events.ON_LOGIN_ANSWER.value, messageOutput);  
    });  
}
```

Paso 8 – Lógica del Logout: Para el **Logout** la cosa es mucho más simple porque no vamos a hacer nada. Simplemente recogemos el `userName` y sacamos una traza. No hay que enviar respuestas siempre al cliente.

```
private DataListener<MessageInput> logout() {
    return ((client, data, ackSender) -> {
        // This time, we simply write the message in data
        System.out.println("Client from " + client.getRemoteAddress() + " wants to logout");

        // The JSON message from MessageInput
        String message = data.getMessage();

        // We parse the JSON into an JsonObject
        // The JSON should be something like this: {"userName": "patata"}
        Gson gson = new Gson();
        JsonObject jsonObject = gson.fromJson(message, JsonObject.class);
        String userName = jsonObject.get("userName").getAsString();

        // We do something on dataBase? _-(ツ)_/

        System.out.println(userName + " logged out");
    });
}
```

Paso 9 – Lógica del GetAll: Finalmente, vamos a hacer el `getAll`. Que, en realidad, es básicamente lo mismo que hemos hecho en el Login, sólo que en este caso devolvemos una lista de Alumnos.

```
private DataListener<MessageInput> getAll() {
    return ((client, data, ackSender) -> {
        // This time, we simply write the message in data
        System.out.println("Client from " + client.getRemoteAddress() + " wants to getAll");

        // We access to database and... we get a bunch of people
        List<Alumno> alumnos = new ArrayList<Alumno>();
        alumnos.add(new Alumno(0, "patata", "potato", "pass", 20));
        alumnos.add(new Alumno(1, "patata2", "potato2", "pass2", 22));
        alumnos.add(new Alumno(2, "patata3", "potato3", "pass3", 23));

        // We parse the answer into JSON
        String answerMessage = new Gson().toJson(alumnos);

        // ... and we send it back to the client inside a MessageOutput
        MessageOutput messageOutput = new MessageOutput(answerMessage);
        client.sendEvent(Events.ON_GET_ALL_ANSWER.value, messageOutput);
    });
}
```

Paso 10 – Lógica del main: Solo queda añadir el código necesario al main () para arrancar todo el tinglado. Añadimos a la clase los métodos de arranque y parada; y después en el main:

```
// Server control
public void start() {
    server.start();
}

public void stop() {
    server.stop();
}

// Server control
public class App {

    private static final String HOST_NAME = "localhost";
    private static final int PORT = 3000;

    public static void main(String[] args) {

        // Server configuration
        Configuration config = new Configuration ();
        config.setHostname(HOST_NAME);
        config.setPort(PORT);

        // We start the server
        SocketIOServer server = new SocketIOServer(config);
        SocketIOModule module = new SocketIOModule(server);
        module.start();
    }
}
```

Ya está listo el server para que realices las pruebas que quieras.

1.4 La App (Cliente)

Tienes un ejemplo creado en la carpeta de ejemplos: **AlumnoClient**. Vamos a detallar aquí los pasos que se han seguido para crearlo desde cero (el ejemplo está más completo todavía).

Paso 1 – Crea el proyecto

Paso 2 – Comprueba el Gradle: Asegúrate de que el Gradle tiene configuradas las siguientes propiedades con los siguientes valores:

- CompileSdk = 35
- MinSDK = 29
- TargetSdk = 35

Ignora el warning indicando que hay problemas en el TargetSdk, o solúcionalo diciendo al Android Studio que sabes lo que haces (probablemente).

Paso 3 – Dependencias: Ahora, añadimos las dependencias:

- Socket.io.client: versión 2.1.1
- Engine.io.client: versión 2.1.0
- Gson: la última versión disponible.

Paso 4 – Permisos: Dale permisos de salida a Internet a la App en el Manifest. Adicionalmente, en este fichero, permite que los mensajes de texto vayan en claro por la red:

```
android:usesCleartextTraffic="true"
```


Paso 5 – El Interfaz: Vamos a empezar creando el Interfaz de la App. Tendremos una serie de botones que lanzarán mensajes, y veremos las respuestas del Server en un TextView y como en las trazas de log. Las funcionalidades de los botones son:

- Connect: Conecta el cliente con el Servidor
- Disconnect: Lo desconecta
- Login: Loguea al usuario en el Servidor.
- Logout: Lo desloguea
- GetAll: Solicita todos los alumnos de BBDD

En este ejemplo estamos haciendo el conectar/desconectar con sendos botones. Esto no tiene sentido en realidad. Tú tienes que ser el que decida cómo, cuándo y por qué un cliente se conecta al Servidor. La app no va a funcionar ni a generar un error si no le das tú a conectar al comenzar a usarla, por mucho que le des a otros botones.

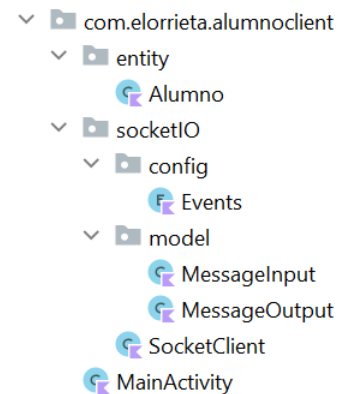


Te habrás dado cuenta de que **cada botón** se corresponde con uno de los **eventos del Server**. Los de conectar y desconectar son el `onConnect ()` y `onDisconnect ()`, mientras que los otros tres se corresponden con `onLogin`, `onLogout` y `onGetAll`.

Paso 6 – Estructura: Generaremos la siguiente estructura de clases:

Si te fijas, son esencialmente las mismas clases que en el lado Servidor. De hecho, las clases **Alumno**, **MessageInput**, **MessageOutput** y **Events** son idénticas, no se ha cambiado una sola coma (salvo pasarlas a Kotlin).

SocketClient dispondrá de los eventos capturados por la app, y se encargará de procesar el envío y recepción de los mensajes. Por último, el **MainActivity** llamará al **SocketClient**



Paso 6 – Entidad, Events, Mensajes: Estas clases son idénticas a las del lado Server. **Alumno** se usará para convertir la información que llega en formato JSON en los mensajes y manipularla más fácilmente, mostrándola en un listado mediante un Adapter.

```
data class Alumno (private val id: Int, private val name: String, private val surname: String,
    private val pass: String, private val edad: Int)
```

Por su parte, **Events** contiene los nombres de los eventos.

```
enum class Events(val value: String) {
    ON_LOGIN ( value: "onLogin"),
    ON_GET_ALL ( value: "onGetAll"),
    ON_LOGOUT ( value: "onLogout"),
    ON_LOGIN_ANSWER ( value: "onLoginAnswer"),
    ON_GET_ALL_ANSWER ( value: "onGetAllAnswer");
}
```

Y **MessageInput** y **MessageOutput** son simplemente:

```
data class MessageInput (private val message: String)
```

Paso 7 – Client: Vamos a añadir código al **SocketClient**. De la misma forma que hicimos con **SocketIOModule** del Server. Añade el bloque **init** a la clase:

```
class SocketClient (private val activity: Activity) {

    // Server IP:Port
    private val ipPort = "http://10.0.2.2:3000"
    private val socket: Socket = IO.socket(ipPort)

    // For log purposes
    private var tag = "socket.io"

    // We add here ALL the events we are eager to LISTEN from the server
    init {
        // Event called when the socket connects
        socket.on(Socket.EVENT_CONNECT) {
            Log.d (tag, msg: "Connected...")
        }

        // Event called when the socket disconnects
        socket.on(Socket.EVENT_DISCONNECT) {
            Log.d (tag, msg: "Disconnected...")
        }
    }
}
```

Paso 8 – Lógica del Login: Le hemos añadido por ahora los dos eventos **Connect** y **Disconnect**. Generarán trazas de log cada vez que se conecte o desconecte un cliente. Vamos ahora con el Login:

```
// Event called when the socket gets an answer from a login attempt.
// We get the message and print it. Note: this event is called after
// We try to login
socket.on(Events.ON_LOGIN_ANSWER.value) { args ->

    // The response from the server is a JSON
    val response = args[0] as JSONObject

    // The answer should be like this:
    // {"id": 0, "name": "patata", "surname": "potato", "pass": "pass", "edad": 20}
    // We extract the field 'message'
    val message = response.getString( name: "message") as String

    // We parse the JSON into an Alumno because... ¯_(ツ)_/¯
    var gson = Gson()
    var jsonObject = gson.fromJson(message, JSONObject::class.java)
    val id = jsonObject["id"].asInt
    val name = jsonObject["name"].asString
    val surname = jsonObject["surname"].asString
    val pass = jsonObject["pass"].asString
    val edad = jsonObject["edad"].asInt

    var alumno = Alumno (id, name, surname, pass, edad)

    // And... me list the Alumno in the list and in the Log
    activity.findViewById<TextView>(R.id.textView).append("\n" + alumno.toString())
    Log.d (tag, msg: "The answer: $alumno")
}
```

Según nuestro propio diseño, el Server nos tiene que enviar como respuesta al login un objeto alumno. Nuestro Servidor es lo que hace: transforma el Alumno en JSON y lo envía dentro de un **MessageInput** al Cliente. Nosotros sólo tenemos que extraerlo, convertir el JSON de vuelta a un objeto Alumno, y utilizarlo.

En este caso hemos decidido parsear el JSON (que contiene un Alumno) a mano, aunque obviamente, podemos dejar a Gson que lo haga él sólo, como en hacemos con el **getAll**.

Paso 9 – Lógica del getAll: Nos quedará así:

```
socket.on(Events.ON_GET_ALL_ANSWER.value) { args ->

    // The response from the server is a JSON
    val response = args[0] as JSONObject

    // The answer should be like this:
    // [
    // {"id":0,"name":"patata","surname":"potato","pass":"pass","edad":20},
    // {"id":1,"name":"patata2","surname":"potato2","pass":"pass2","edad":22},
    // {"id":2,"name":"patata3","surname":"potato3","pass":"pass3","edad":23}
    // ]
    // We extract the field 'message'
    val message = response.getString( name: "message") as String

    // We parse the JSON. Note we use Alumno to parse the server response
    var gson = Gson()
    val itemType = object : TypeToken<List<Alumno>>() {}.type
    var list = gson.fromJson<List<Alumno>>(message, itemType)

    // The logging
    activity.findViewById<TextView>(R.id.textView).append("\nAnswer to getAll:$list")
    Log.d(tag, msg: "Answer to getAll: $list")
}
```

Paso 10 – Lógica del Activity: Por ahora, nuestro Cliente simplemente escucha de forma pasiva al Server. Vamos a darle funcionalidad. Añadiremos fuera del init los siguientes métodos:

```
// This method is called when we want to establish a connection with the server
fun connect() {
    socket.connect()

    // Log traces
    activity.findViewById<TextView>(R.id.textView).append("\n" + "Connecting to server...")
    Log.d(tag, msg: "Connecting to server...")
}

// This method is called when we want to disconnect from the server
fun disconnect() {
    socket.disconnect()

    // Log traces
    activity.findViewById<TextView>(R.id.textView).append("\n" + "Disconnecting from server...")
    Log.d(tag, msg: "Disconnecting from server...")
}
```

Estos métodos conectarán y desconectarán nuestro cliente al servidor. Ahora, añadiremos también las tres operaciones básicas que hemos diseñado: **login**, **getAll** y **logout**.

```

// This method is called when we want to login. We get the userName,
// put in into an MessageOutput, and convert it into JSON to be sent
fun doLogin(userName: String) {
    val message = MessageInput(userName) // The server is expecting a MessageInput
    socket.emit(Events.ON_LOGIN.value, Gson().toJson(message))

    // Log traces
    activity.findViewById<TextView>(R.id.textView).append("\nAttempt of login - $message")
    Log.d (tag, msg: "Attempt of login - $message")
}

// This method is called when we want to getAll the Alumno.
fun doGetAll() {
    socket.emit(Events.ON_GET_ALL.value)

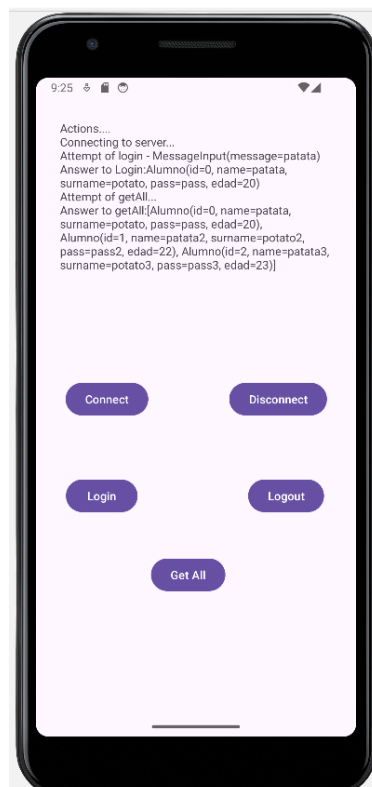
    // Log traces
    activity.findViewById<TextView>(R.id.textView).append("\nAttempt of getAll...")
    Log.d (tag, msg: "Attempt of getAll...")
}

// This method is called when we want to logout. We get the userName,
// put in into an MessageOutput, and convert it into JSON to be sent
fun doLogout(userName: String) {
    val message = MessageInput(userName) // The server is expecting a MessageInput
    socket.emit(Events.ON_LOGOUT.value, Gson().toJson(message))

    // Log traces
    activity.findViewById<TextView>(R.id.textView).append("\nAttempt of Logout - $message")
    Log.d (tag, msg: "Attempt of Logout - $message")
}

```

Prueba la App. El resultado debería de ser algo como esto:



Una serie de cosas sobre este ejemplo de Socket.io.

- No he capturado ninguna excepción ni en el Cliente ni en el Server. No lo he hecho por simplicidad.
- Si el Cliente no conecta con el Servidor, nadie te va a avisar del error. Lo mismo pasa si hay una excepción en el servidor. La App va a quedarse 'tostada' esperando indefinidamente sin dar error. Lo conveniente es que tengas un Servidor y un Cliente con trazas de log y que hagas uso de paciencia.

Ejercicio 99

Crea una App y un servidor que sean capaces de proporcionar la siguiente funcionalidad. El centro Elorrieta-Errekamari dispone de una base de datos con los nombres de los alumnos y sus números de teléfono. Desarrolla una app Android en Kotlin que sea capaz de consultar el servidor y gestionar dicha base de datos.

Operaciones y sus posibles escenarios:

- Buscar teléfono por nombre, si existe, se muestra. Si no existe, se informa al usuario.
- Añadir teléfono al alumno
- Crear alumno. Informar si el alumno ya existe
- Borrar alumno por nombre. Informar si no existe
- Modificar alumno. Informar si no existe

La información que vamos a guardar de alumno es: id, nombre, apellido, teléfono. Dado que es un ejercicio simple, no hace falta que añadas una base de datos real; basta con que la simules con un simple ArrayList precargado durante el arranque en el servidor.

Ejercicio 99

Modifica el ejercicio anterior para que permita mostrar la **foto** del alumno almacenada en la base de datos del servidor. Adicionalmente, dado que el número de teléfono es información sensible, tiene que viajar de forma **segura** por la red. El teléfono se cifra mediante clave **público-privada**. El fichero de clave privada utilizado para descifrarlo en tu App se añade manualmente en la carpeta res/raw.

Consulta los apuntes sobre seguridad y cifrado de la asignatura de **PSP**.

Ejercicio 99

Modifica el ejercicio anterior garantizar que todo intercambio de mensajes entre cliente y servidor, en ambas direcciones, se cifra mediante clave **público-privada**. Por tanto, necesitarás DOS juegos de claves. Adicionalmente, ha que ambas Claves Públicas se obtengan dinámicamente de sus respectivos, sin tener que almacenarlas manualmente.

Consulta los apuntes sobre seguridad y cifrado de la asignatura de **PSP**.